

HUTECH Institute of Artificial Intelligence &
Digital Transformation (HUTECH-AIDT)

Week 4: Topological Data Analysis (TDA) & Deep Learning (DL)

HUTECH AI Research Internship Program 2026



Topological Data Analysis (TDA) & Deep Learning (DL) :

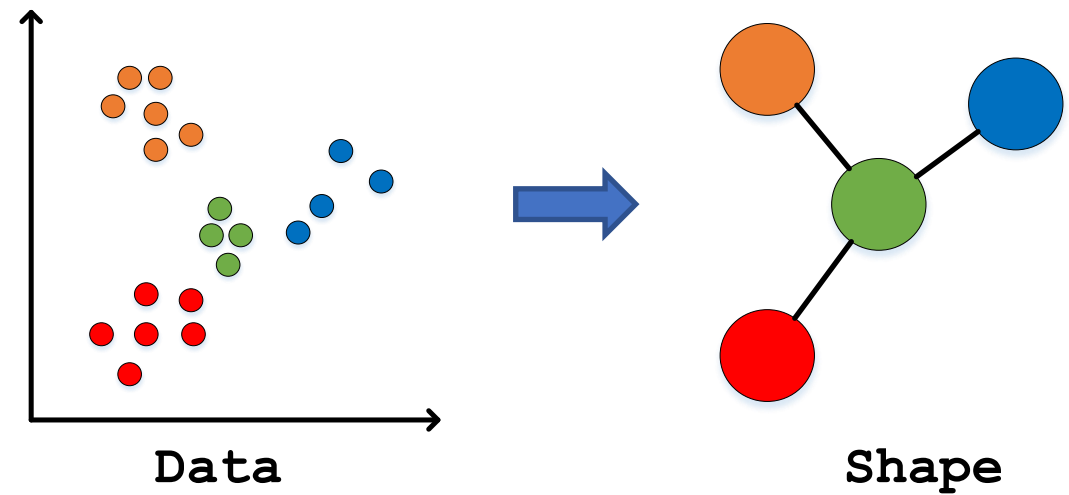
- Introduction to TDA concepts
- Persistent homology
- Persistence diagrams and persistence images
- Topological representation learning
- Applications of TDA in AI and graph learning.

Introduction to topological data analysis (TDA)

What is Topological Data Analysis?

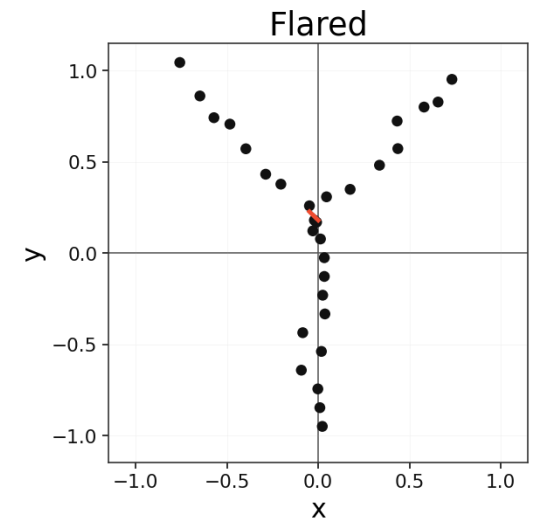
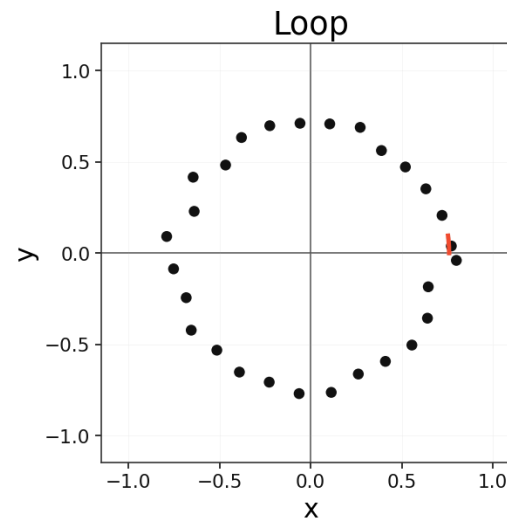
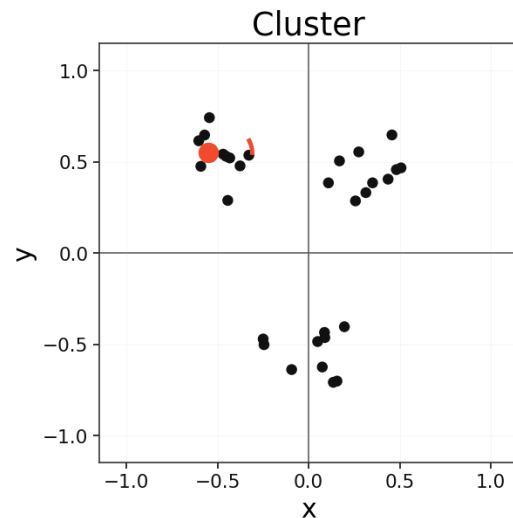
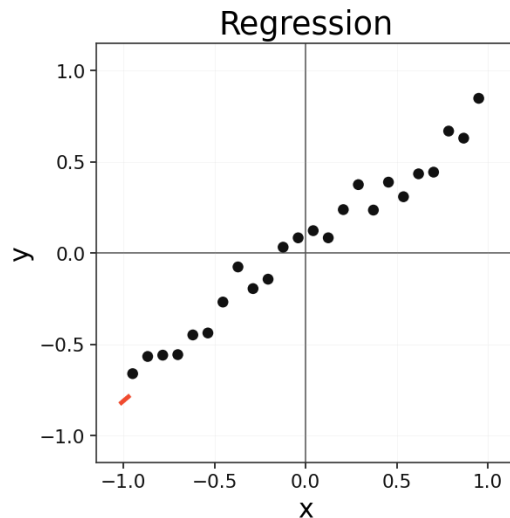
- A **mathematical framework** for analyzing the shape and structure of data.
- Extracts **geometric and topological properties** beyond conventional numerical features.
- Captures **global relationships, connectivity, loops, and higher-order structures** within **complex datasets**.
- Provides complementary information to traditional ML and DL approaches.

The data has shape and the shape is important.



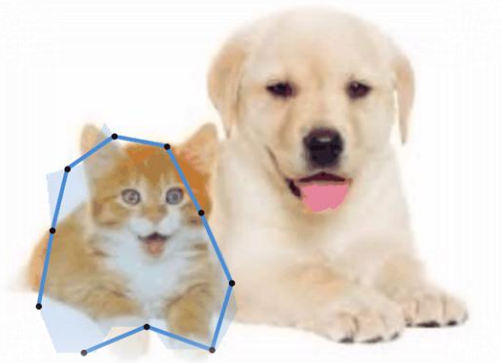
Why geometry and topology matter in AI?

- Many real-world datasets contain hidden **geometric structures** and **spatial relationships**.
- Data often lies on **complex manifolds** rather than **simple Euclidean spaces**.
- **Structural information** can **reveal patterns** not captured by statistical features alone.
- Understanding **data shape** can improve **robustness, interpretability,** and **representation learning**.



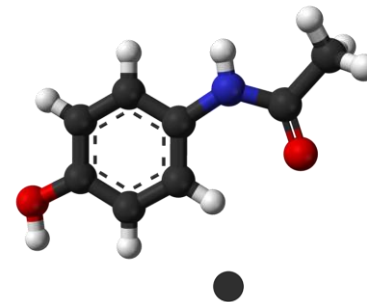
TDA approach : shapes in data

Traditional Statistics	Topological Analysis
Focuses on numerical values	Focuses on shape and structure
Mean, variance, correlation	Connected components, loops, cavities
Sensitive to local perturbations	Robust to deformation and noise
Captures local properties	Captures global structural information

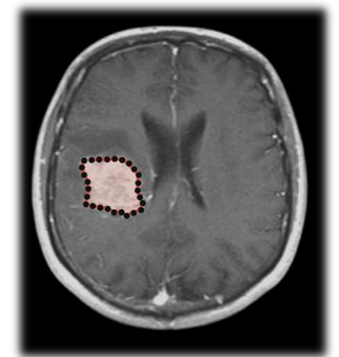


Images → object shapes and contours.

Graphs → communities and connectivity patterns.



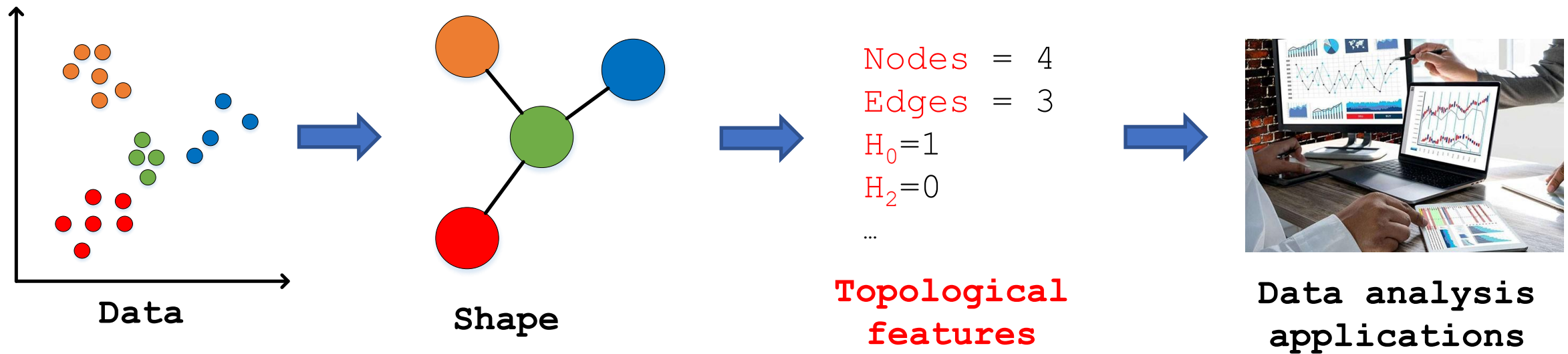
Molecules → ring structures and chemical bonds.



Medical images → anatomical structures and lesions

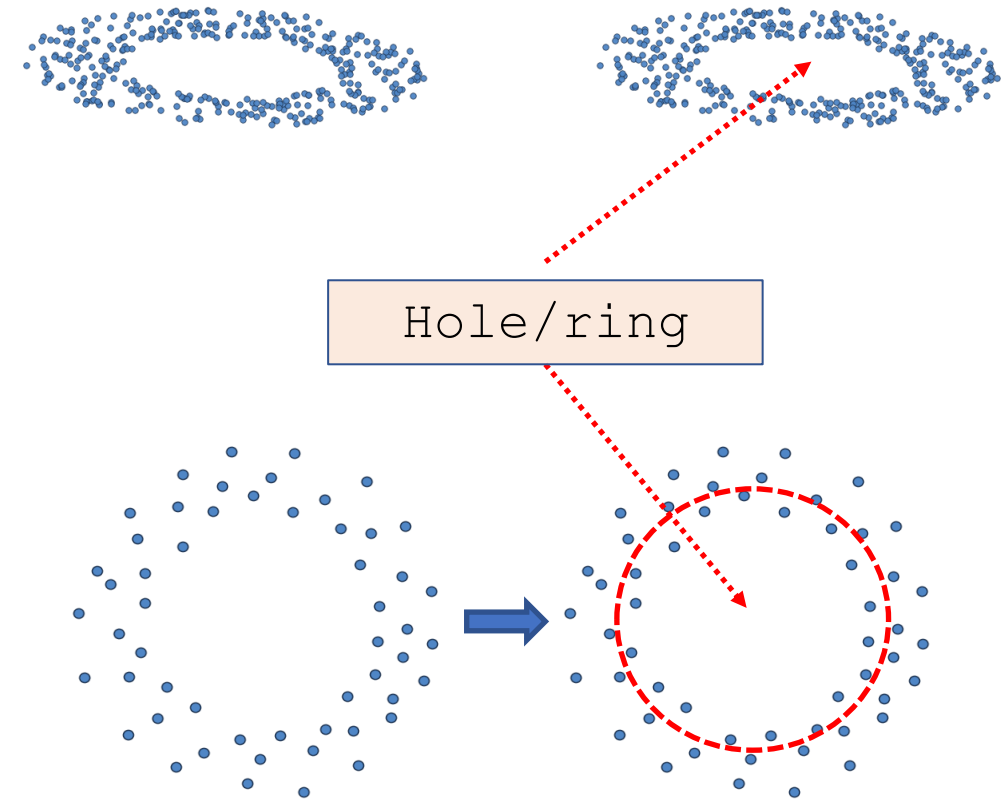
General pipelines for data analysis with TDA

- **Transform** the **data to shape**, such as **graph** (e.g., Euler method, etc.)
- Extracting **topological features** from **data shape**.
- Using these **topological features** for different data analysis problems.

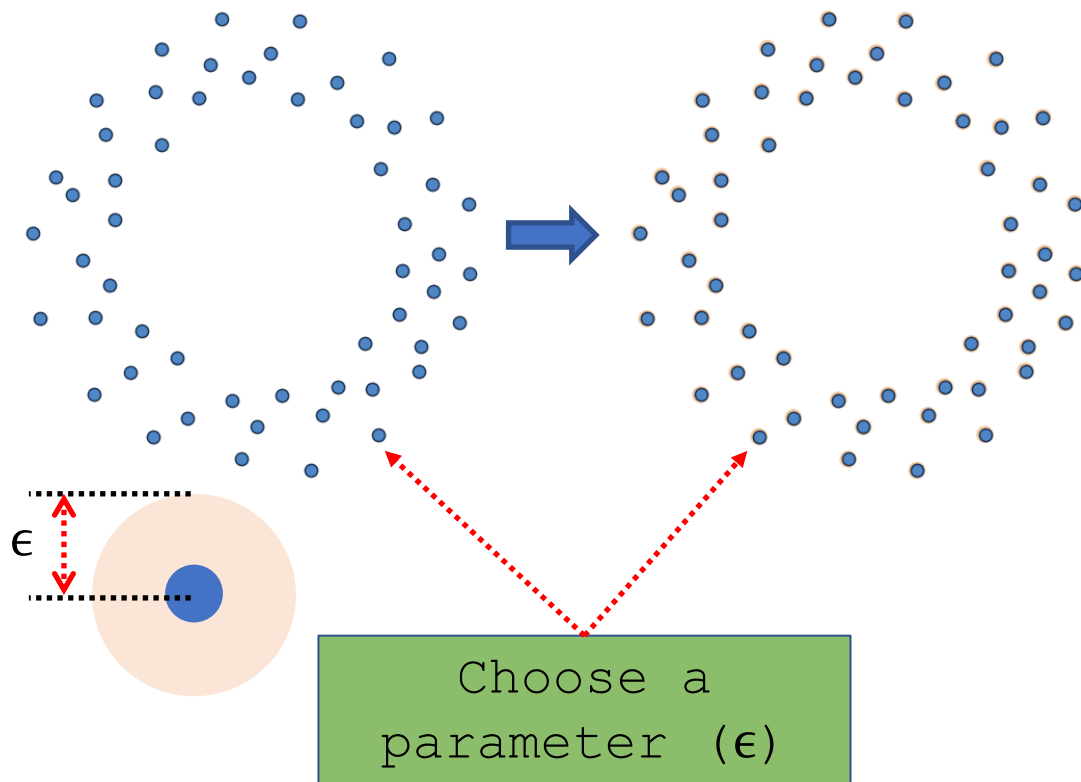


Topological features of data

- How the **topological features** of **data** can be evaluated & extracted?
- **Nearby data points** are **connected** using a **predefined distance threshold** to reveal meaningful **structural patterns**.
- **Loops and rings** represent important **structural characteristics** that cannot be captured by traditional statistical methods.
- **Persistent topological structures** often correspond to **meaningful patterns**, while **short-lived structures** are usually **noise**.



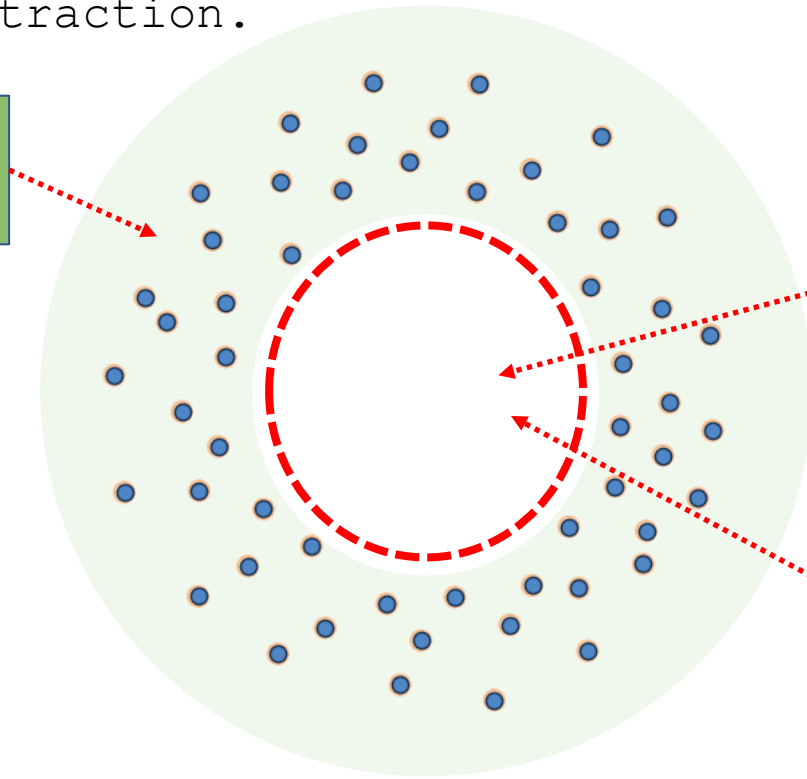
All data points have a surrounding circle with (d) diameter



- Each **data point** is associated with a **neighborhood region** defined by a selected **distance threshold (d)** .
- The **distance parameter** controls how **local geometric relationships** are established among neighboring samples.
- Small **distance values** preserve **isolated points** and **disconnected components** within the underlying data structure.
- Increasing the **distance threshold** gradually connects nearby points and reveals larger **topological structures** and patterns.
- Overlapping **neighborhoods** create **connected regions** that enable the identification of **loops** and **holes**.

- Connecting **two data points** if their (ϵ) -parameter surrounding circles are **intersected**.
- This empty region corresponds to a potential **topological hole** that cannot be described by cluster analysis alone.
- Detecting such **holes and loops** motivates the development of **Persistent Homology** and topological feature extraction.

Forming a single cluster of data points



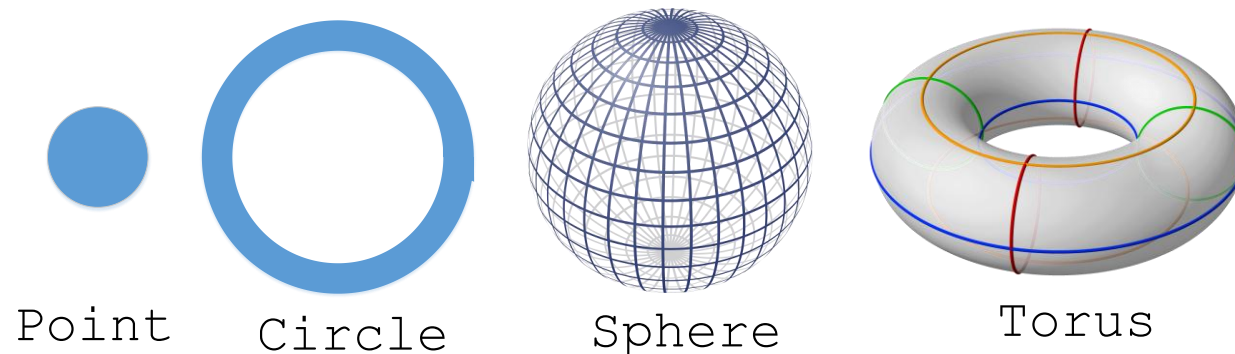
How can we know this hole is a significant topological feature ?

But how can this hole can be identified ?

Betti (β) number: counting the number of shapes (e.g., holes) in **different dimensions** that occur in an object. For examples:

- β_0 : connected components.
- β_1 : holes.
- β_2 : voids.
- ...

Topological Space	β_0	β_1	β_2
Point	1	0	0
Circle	1	1	0
Sphere	1	0	1
Torus	1	2	1

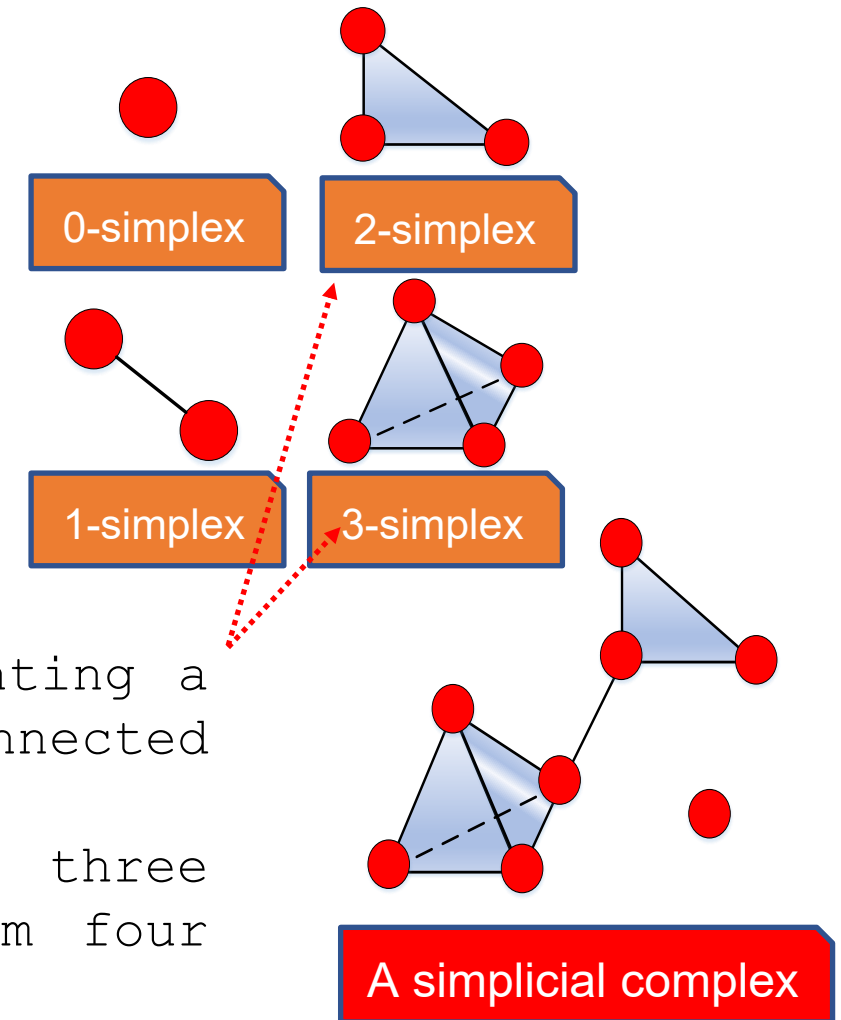


Betti (β) number (con't): the (k^{th}) Betti number (β_k) is the rank of k^{th} **homology group** of topological space ($\mathbf{X}$), denoted as: **$H_k(\mathbf{X})$** .

For example:

- ❖ β_0 : connected components $\rightarrow H_0$
- ❖ β_1 : tunnels/circles $\rightarrow H_1$
- ❖ ...

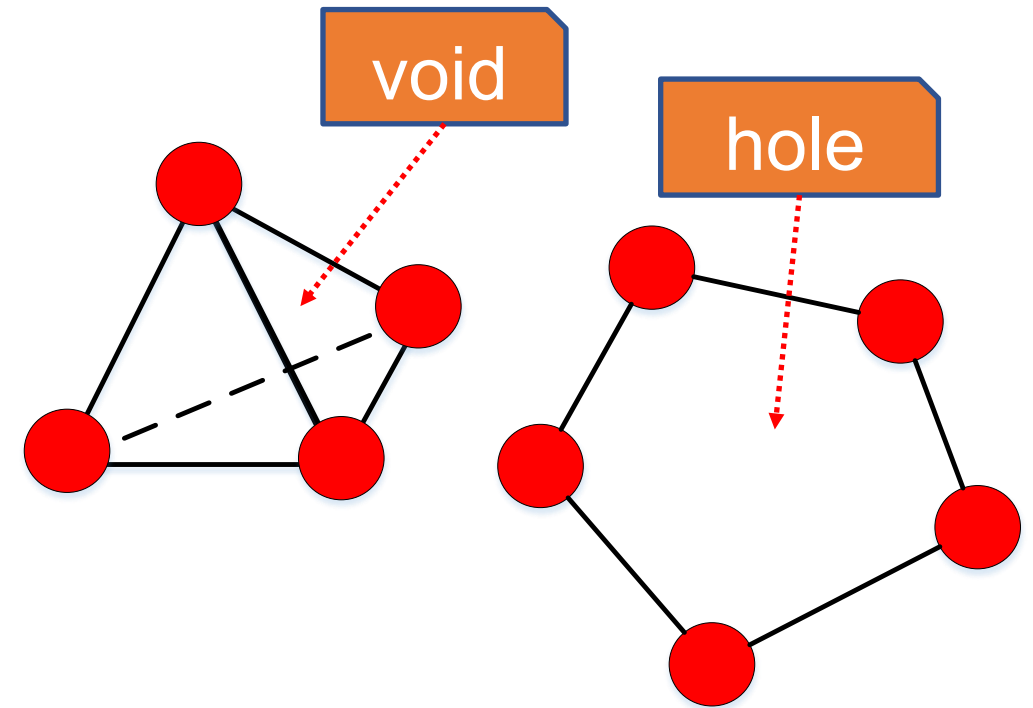
- A **simplicial complex** is built from data **points**, **edges** between data points, **triangular faces**, etc. It's a **collection** of **simplices**.
- Simplices are the **building blocks**, while a simplicial complex provides a **topological representation** of the entire dataset.
- A **2-simplex** forms a **filled triangle**, representing a higher-order interaction among three connected points.
- A **3-simplex** extends the concept into three dimensions, forming a **tetrahedral** volume from four vertices.



Homology of simplicial complex

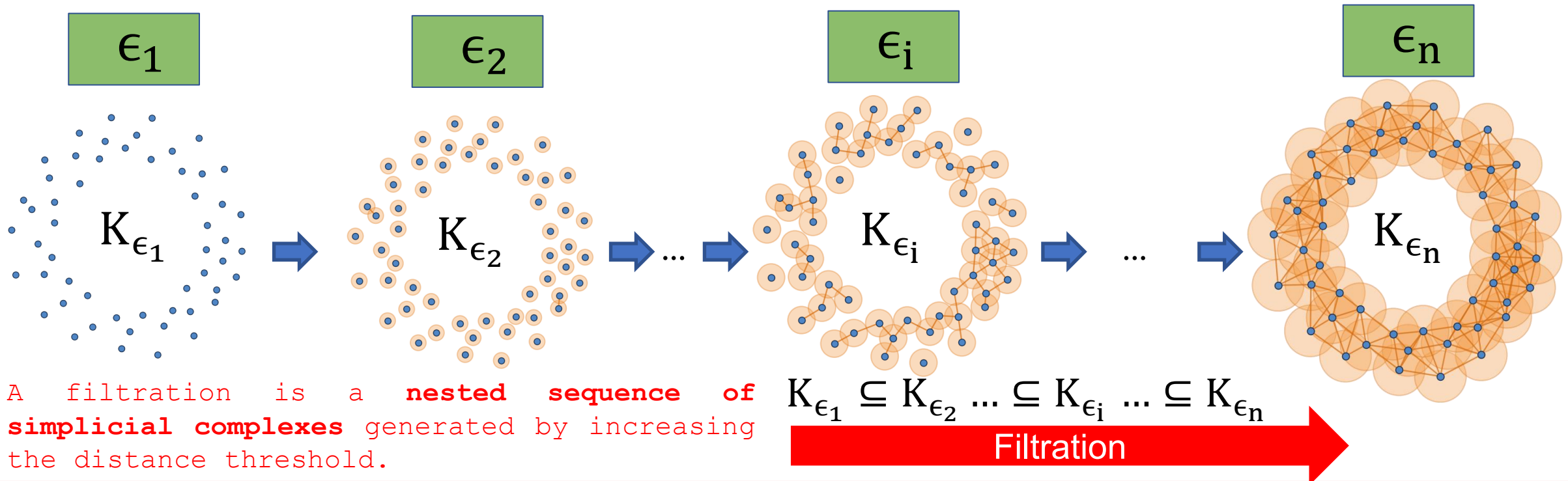
When many simplices are combined, they form a **simplicial complex**, which serves as the mathematical structure used by **Persistent Homology** to identify:

- Connected components (β_0)
- Loops or holes (β_1)
- Cavities or voids (β_2)
- **Homology** counts **components, holes, voids, etc. (shapes)** in the given **data/topological space**.
- **Homology** of a simplicial complex is the number of shapes that occur within it.



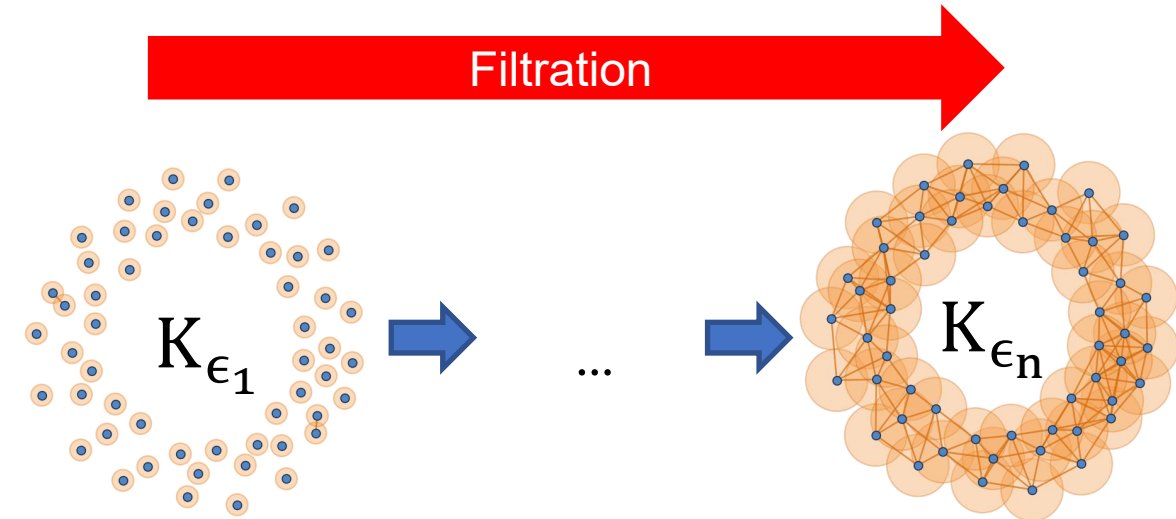
Filtration learning process

- The below **sequence of simplicial complexes**, with **inclusion maps**, this process is called as: **filtration**.
- At small scales, **data points** remain isolated and form multiple disconnected **topological components**.
- Increasing the **filtration parameter** creates new **edges** and progressively connects neighboring data points.



Filtration learning process (con't)

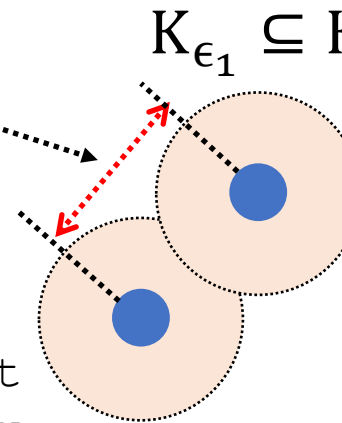
- The below **sequence** of **simplicial complexes**, with **inclusion maps**, this process is called as: **filtration**.
- At small scales, **data points** remain isolated and form multiple disconnected **topological components**.
- Increasing the **filtration parameter** creates new **edges** and progressively connects neighboring data points.



Two data points are connected when their pairwise distance is smaller than the filtration threshold:

$$d(x_i, x_j) \leq \epsilon$$

Filtration analyzes how the topology of a dataset evolves across multiple scales by **continuously increasing** the connectivity threshold (ϵ) .



$$K_{\epsilon_1} \subseteq K_{\epsilon_2} \dots \subseteq K_{\epsilon_i} \dots \subseteq K_{\epsilon_n}$$

- (ϵ) is the filtration parameter.
- (K_{ϵ_i}) : simplicial complex at (ϵ_i) threshold.

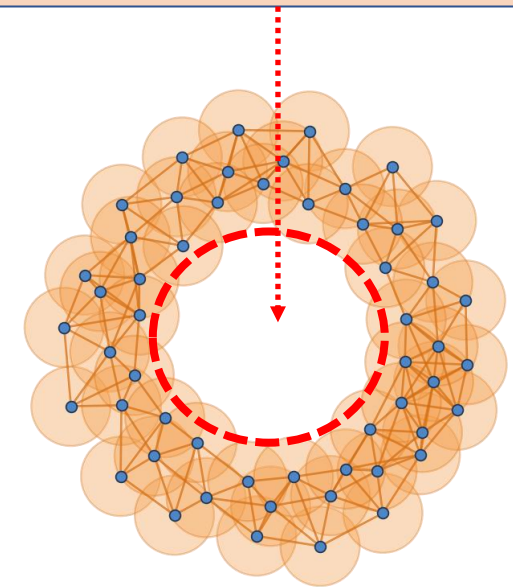
- From found simplices $\rightarrow$ obtain the **Rips (or: Vietoris-Rips) (VR) complex** $\rightarrow$ **identifying topological features** (e.g., holes from the given topological space) ^[*].
- Given **point cloud (X)**, a **simplex** is added to the **simplicial complex** when **all pairs of its vertices** satisfy the **distance constraint**:

$$VR_{\epsilon}(X) = \{\sigma \subseteq X \mid d(v_i, v_j) \leq \epsilon, \forall v_i, v_j \in \sigma\}$$

Interpretation:

- Small** (ϵ) produces **isolated points**.
- Larger** (ϵ) creates edges, triangles, and higher-dimensional simplices.
- The topology evolves as: (ϵ) increases.

Applying homology on this Rips complex to identify the central hole

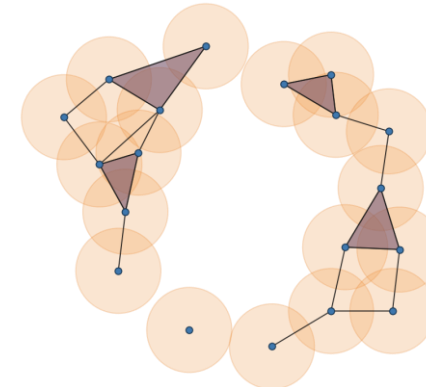


^[*]Carlsson, E., Carlsson, G., & De Silva, V. (2006). An algebraic topological method for feature identification. International Journal of Computational Geometry & Applications, 16(04), 291-314.

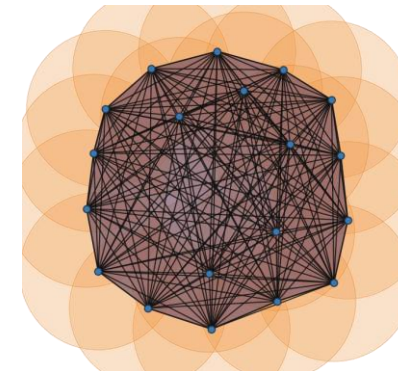
Finding completed simplices (con't)

- The problem is how to choose the **proper value of (ϵ) value**.
- Large filtration values may **over-connect data points**, causing meaningful **holes** and **loops** to disappear.

→ Rather than selecting one threshold, **Persistent Homology** analyzes **topological features** across all **filtration scales** → the **Persistent Barcode & Persistent Diagram** records when features are born and die, helping distinguish meaningful structures from noise.



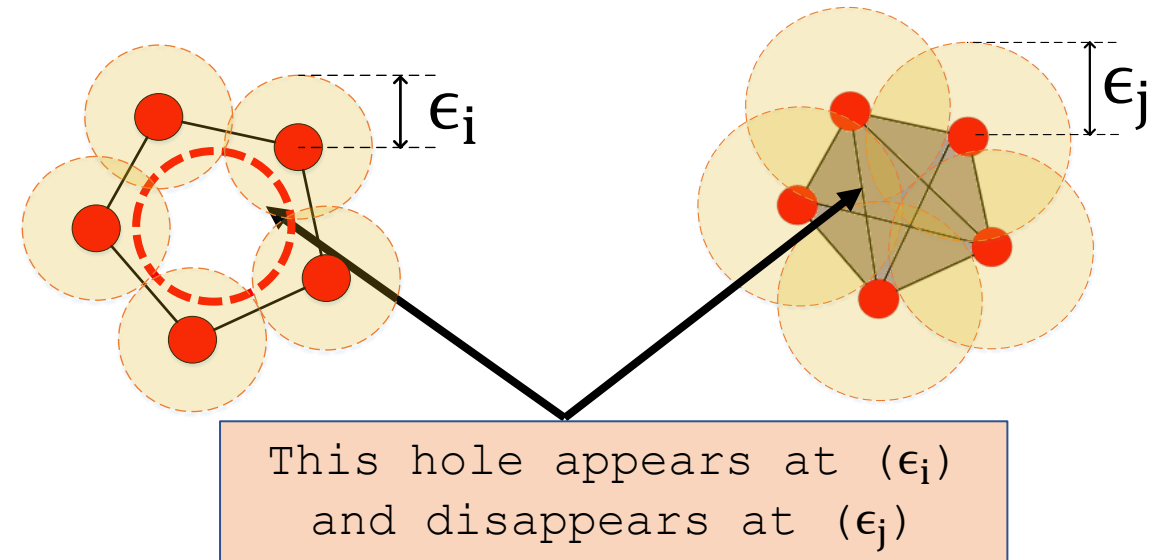
The (ϵ) value is too small → detecting a lot of noises



The (d) value is too large → getting a giant simplex (trivial homology)

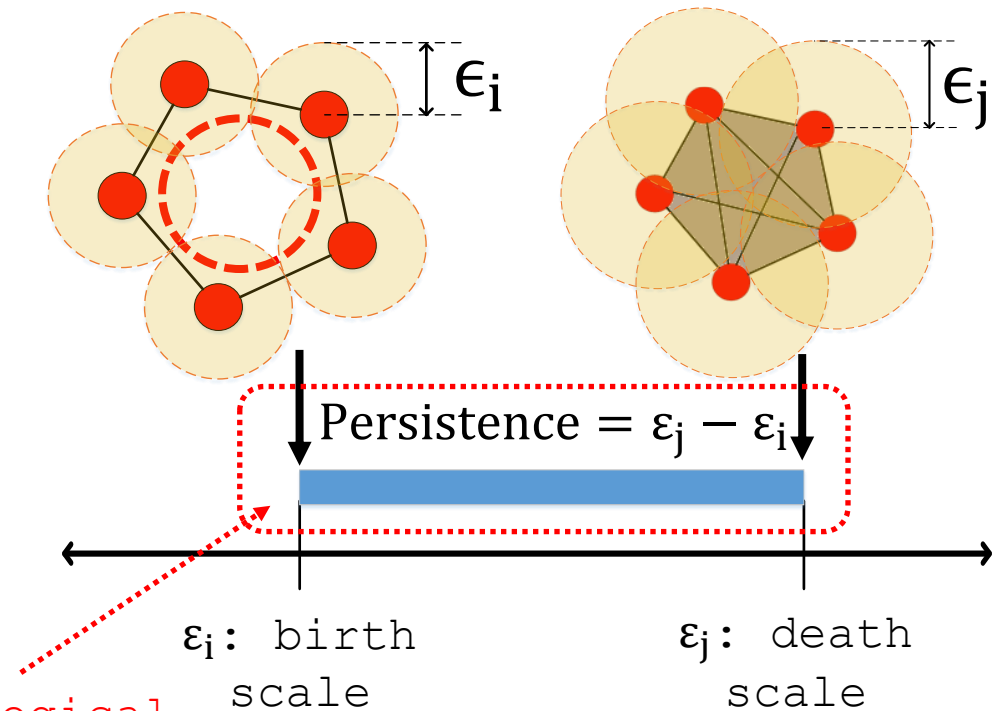
Persistence homology

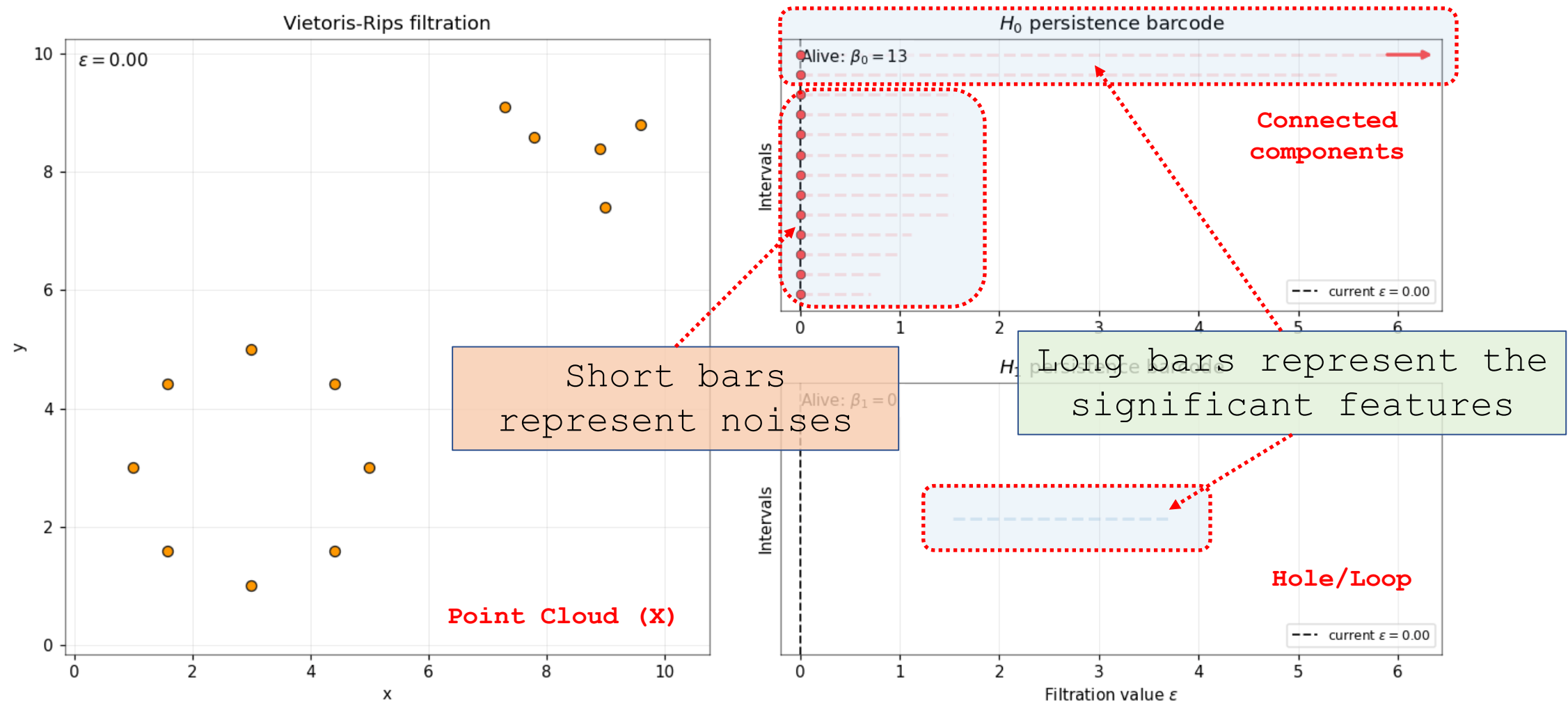
- During filtration, topological features such as **holes** may **appear** and **disappear** at different **filtration scales**.
- The **birth value** (ϵ_i) corresponds to the filtration scale where a hole first emerges.
- The **death value** (ϵ_j) represents the scale where the hole becomes filled by higher-dimensional simplices.
- Each topological feature** is characterized by a **birth-death pair** (ϵ_i, ϵ_j) that summarizes its lifetime.



- Features that **persist across a wide filtration range** $\rightarrow$ considered **meaningful structural patterns**.
- Short-lived features** $\rightarrow$ noise & limited topological significance.

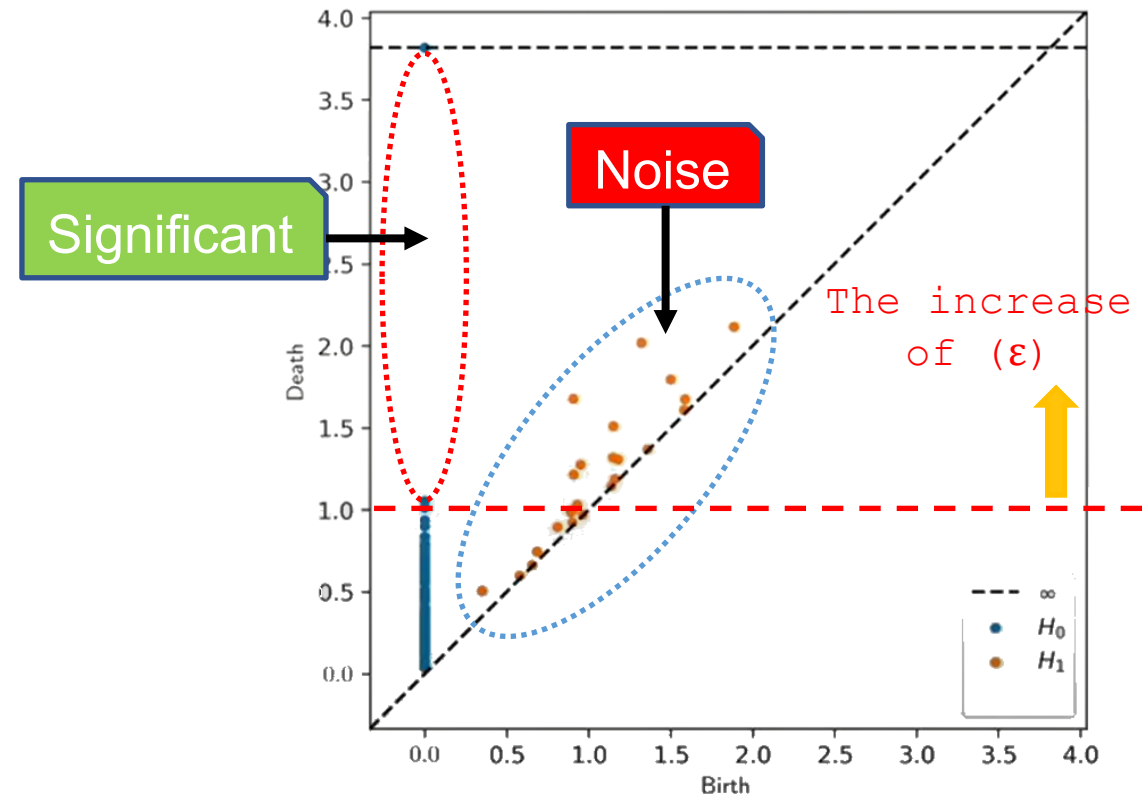
- The **persistence** of **topological features** within the pair of (ϵ_i, ϵ_j) can be visualize as a **blue bars**.
- The collection of bars is a **barcode**.
- Each horizontal bar represents a **single feature characterized** by its **birth** and **death filtration values**.
- **Long bars** $\rightarrow$ persistent and meaningful topological structures that remain stable across scales.
- **Short bars** $\rightarrow$ usually represent noisy features that exist only within a narrow filtration range.





Persistent diagram: representing persistent homology

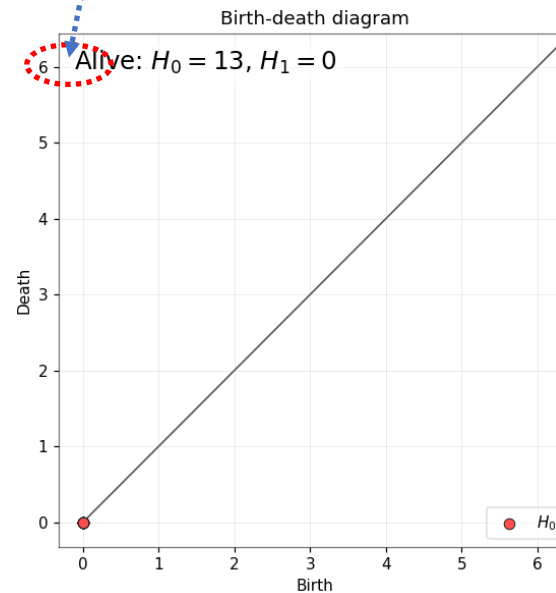
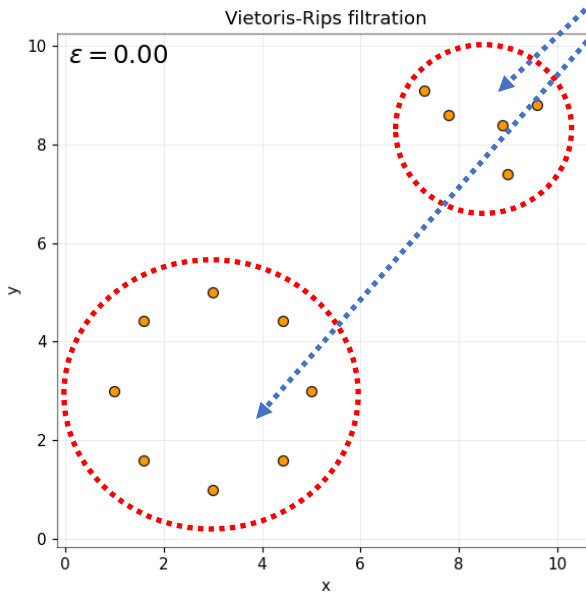
- A **persistence diagram** represents each topological feature as a point using its birth and death scales.
- The horizontal axis records the **birth value**, while the vertical axis records the **death value**.
- **Each point** corresponds to a **topological feature** identified during the filtration process.



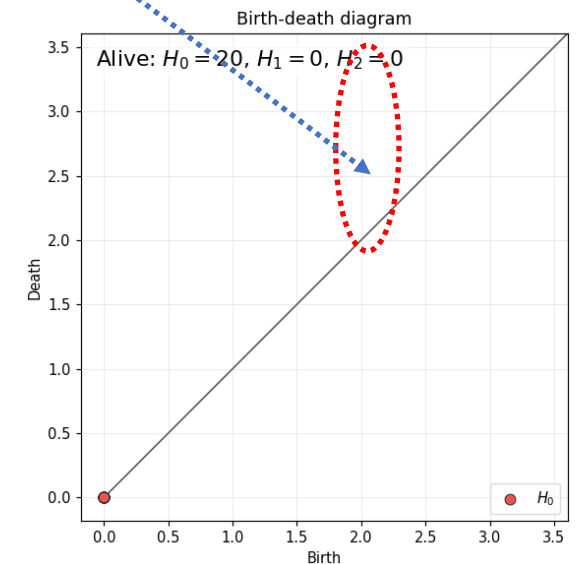
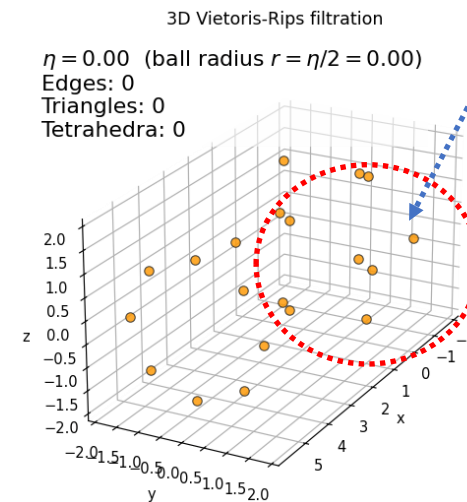
- Features located **far** from the **diagonal** → indicate **meaningful** and **persistent topological structures**.
- Points **near** the **diagonal** usually correspond to **short-lived** features and are often **considered noise**.

Persistent diagram: representing persistent homology (con't)

The last one died out - two components joined together



A three-dimensional void appears within the enclosed structure.

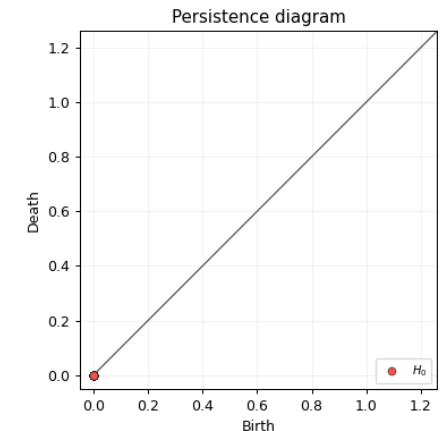
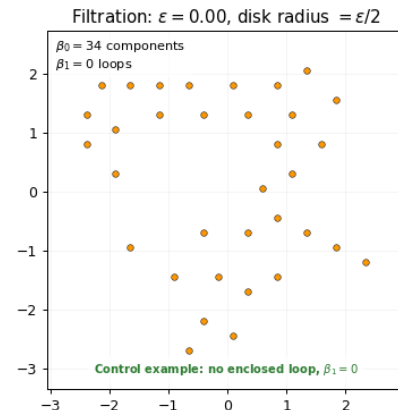


Persistent diagram: representing persistent homology (con't)

MNIST Digit "7" (No Loop)

- The digit 7 **does not** contain any **enclosed region** or **closed topological loop**.
- During **filtration**, **connected components** gradually merge, but **no persistent H_1 feature** is created.

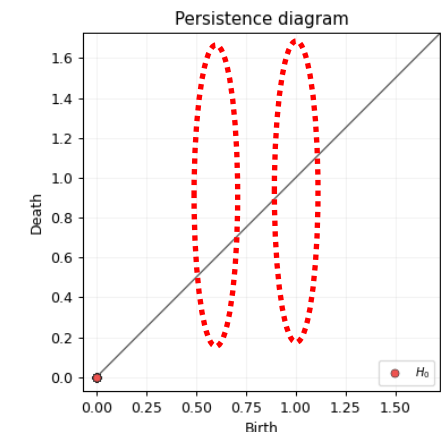
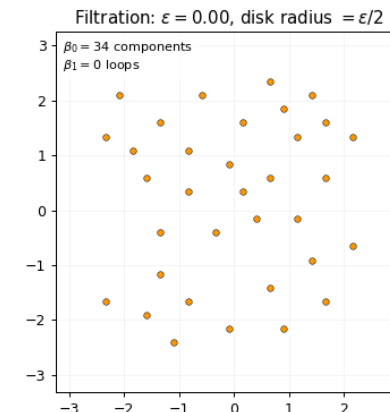
Original MNIST digit 7: 0 enclosed image loops



MNIST Digit "8" (Two Loops)

- The digit 8 contains two enclosed regions, generating **two distinct topological loops**.
- During filtration, these loops appear as **persistent H_1 features** and survive **across multiple scales**.

Original MNIST digit 8: 2 enclosed image loops



Persistence images: representing persistent homology

- Persistence Images transform **variable-sized topological features** into **fixed-dimensional representations** suitable for ML/DL models.
- Each point is transformed into a **Gaussian kernel** on a continuous surface.

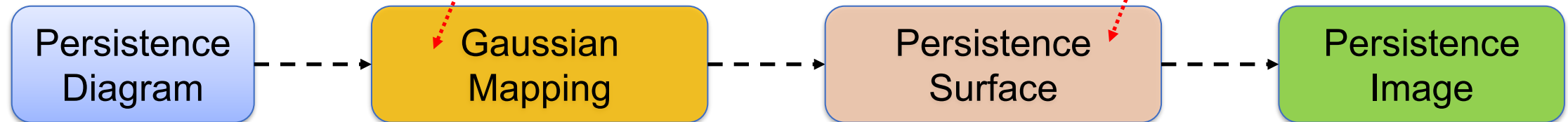
Each point in persistence diagram:

(b_i, p_i)

$$G_i(x, y) = w_i \exp\left(-\frac{(x - b_i)^2 + (y - p_i)^2}{2\sigma^2}\right)$$

All Gaussian kernels are **aggregated** to construct a **continuous persistence surface**:

$$\rho(x, y) = \sum_{i=1}^N G_i(x, y)$$



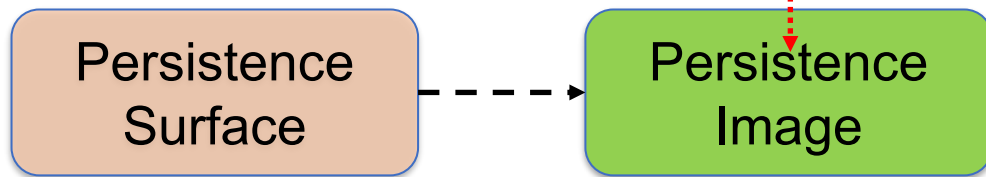
- b_i : birth value
- p_i : persistence value
- σ : Gaussian bandwidth
- w_i : weighting coefficient

- $\rho(x, y)$ denotes the persistence surface.
- N : is the number of topological features extracted from the persistence diagram.

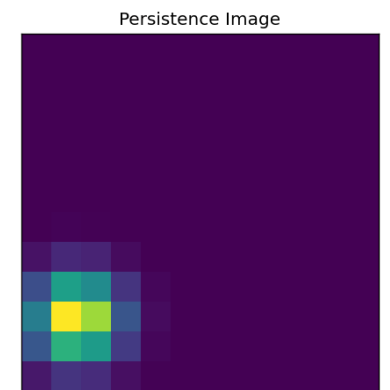
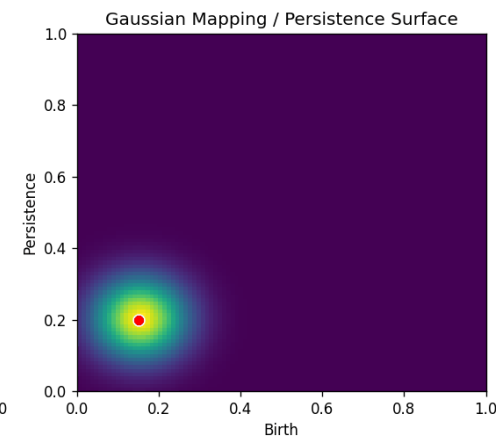
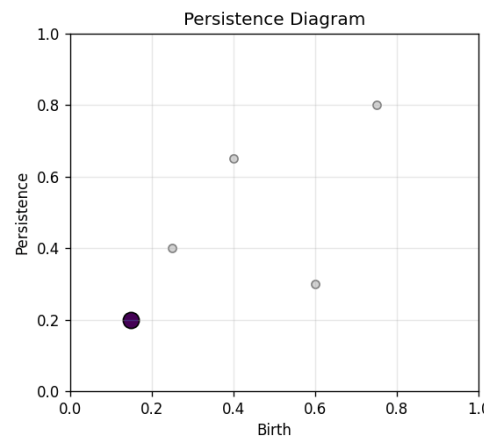
Persistence images: representing persistent homology (con't)

The persistence surface is discretized into a fixed-size grid to generate the persistence image:

$$PI_{m,n} = \iint_{\text{Cell}_{m,n}} \rho(x, y) dx dy$$



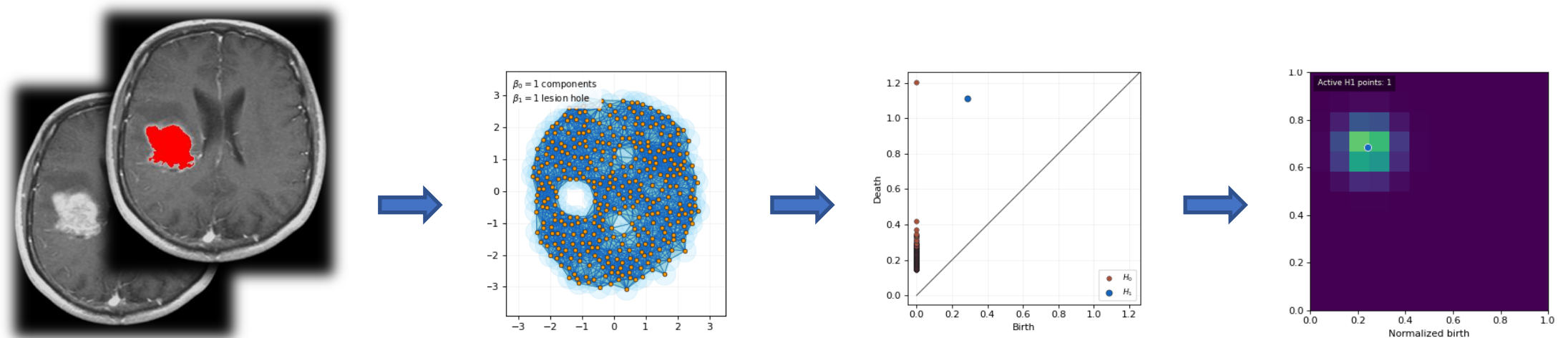
- $PI_{m,n}$ denotes the pixel value at grid cell (m, n) .
- Each pixel **stores** the **accumulated topological importance** within **that region**.



Persistence Diagram → Gaussian Surface → Persistence Image

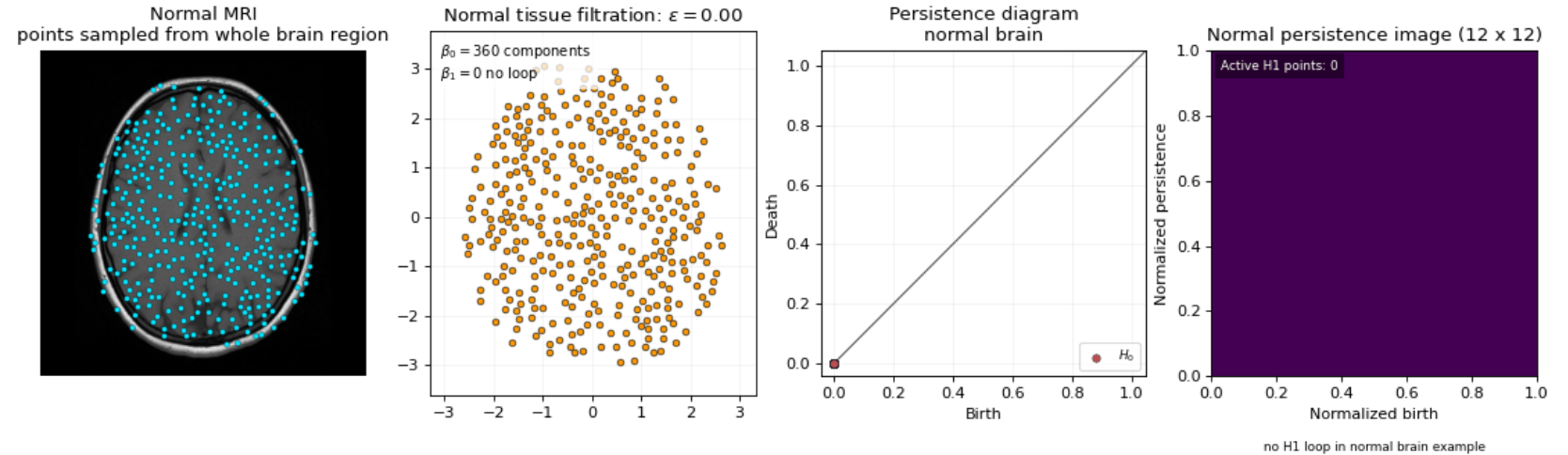
Persistence images: representing persistent homology (con't)

- TDA first converts the segmented tumor region into a **point cloud** representing the underlying geometric structure.
- **Each point** is expanded through **filtration**, gradually revealing **connectivity patterns** among **neighboring tumor pixels**.
- Connected regions form **H_0 features**, describing the overall connectivity of the tumor structure.
- Enclosed **empty regions generate H_1 features**, capturing holes and ring-like topological patterns within the **lesion**.

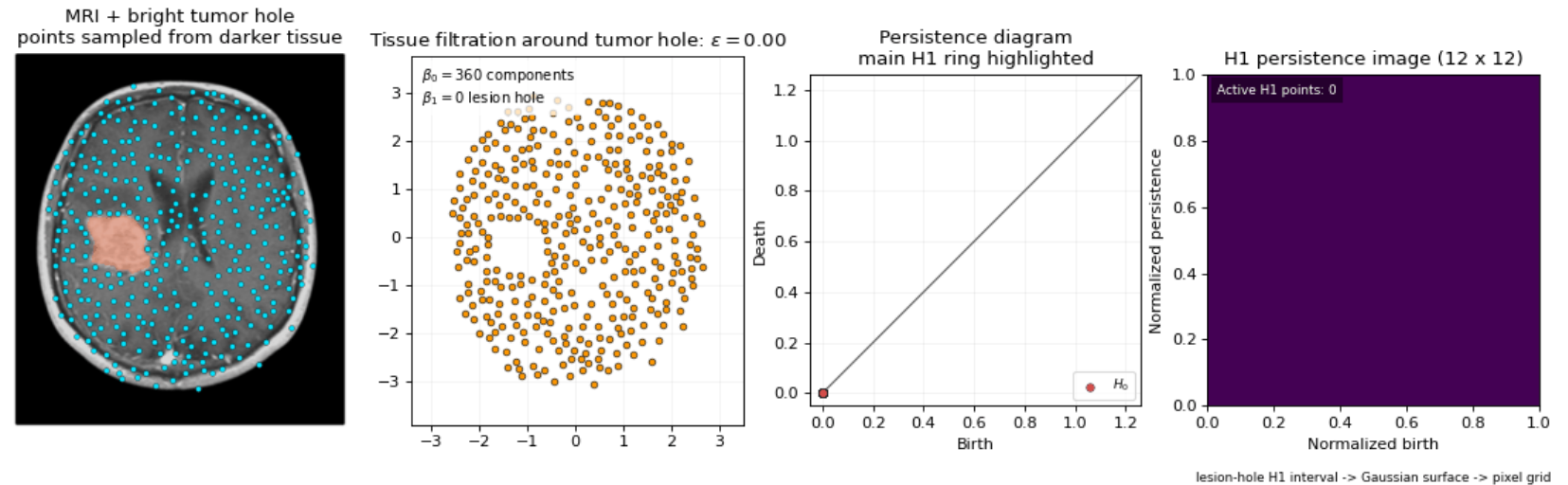


Persistence images: representing persistent homology (con't)

Normal brain tissue exhibits **no significant H_1 loops**, resulting in sparse topological representations.



Tumor regions create **persistent H_1 features**, producing distinctive patterns in persistence diagrams and images.



TopoDL: TDA + deep learning

Traditional DL models learn from:

- **Pixel intensities** in images [1].
- **Node features** in graphs [2].
- **Sequential patterns** in time-series [3].

→ However, they may overlook important **global topological structures** such as **connectivity, loops**, and higher-order relationships.

→ TDA provides complementary **structural information** that can enhance representation learning.

[1] Singh, Y., Farrelly, C. M., Hathaway, Q. A., Leiner, T., Jagtap, J., Carlsson, G. E., & Erickson, B. J. (2023). Topological data analysis in medical imaging: current state of the art. *Insights into Imaging*, 14(1), 58.

[2] Pham, P., Bui, Q. T., Nguyen, N. T., Kozma, R., Yu, P. S., & Vo, B. (2025). Topological data analysis in graph neural networks: Surveys and perspectives. *IEEE Transactions on Neural Networks and Learning Systems*, 36(6), 9758–9776.

[3] Zia, A., Khamis, A., Nichols, J., Tayab, U. B., Hayder, Z., Rolland, V., ... & Petersson, L. (2024). Topological deep learning: a review of an emerging paradigm. *Artificial Intelligence Review*, 57, 77.

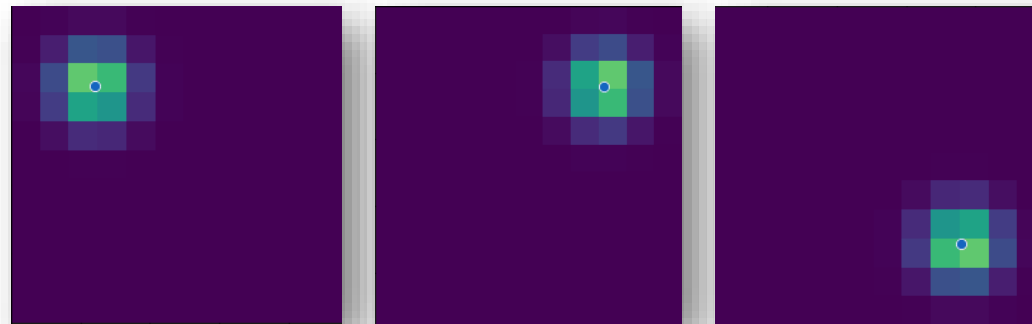
TopoDL: topological invariant encoding

Unlike raw features, **topological representations encode:**

- Connected components (β_0)
- Loops and holes (β_1)
- Cavities and voids (β_2)

These characteristics are **topological invariants**, meaning they remain unchanged under:

- Translation
- Rotation
- Scaling
- Moderate deformation
- Small perturbations

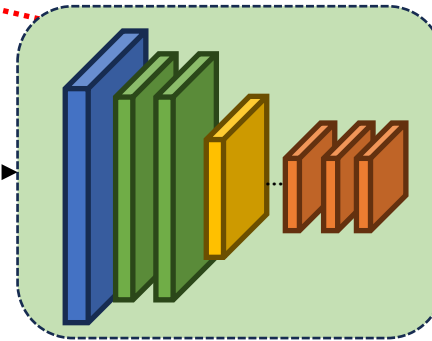


Topology remains stable under translation
of geometric structures

TopoDL: general workflow

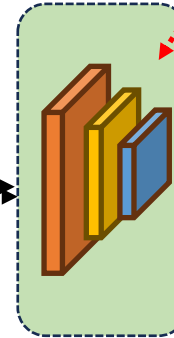
Semantic encoder learns discriminative visual patterns from images

Semantic image encoding

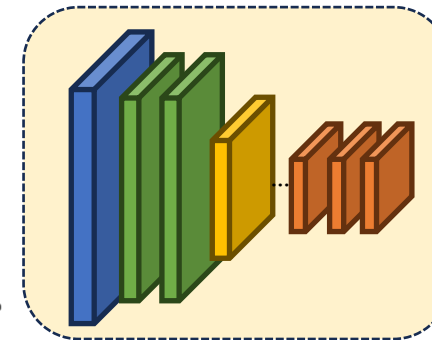


Fused features improve downstream prediction performance and robustness.

Fusion mechanism

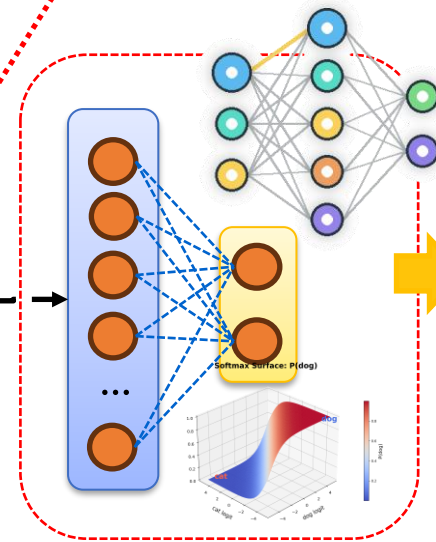


Topological invariant encoding



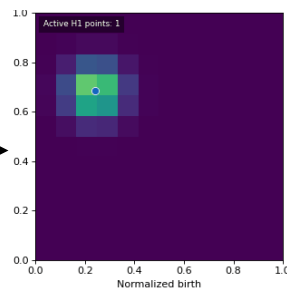
Topological encoder extracts robust invariant shape representations.

Task layer

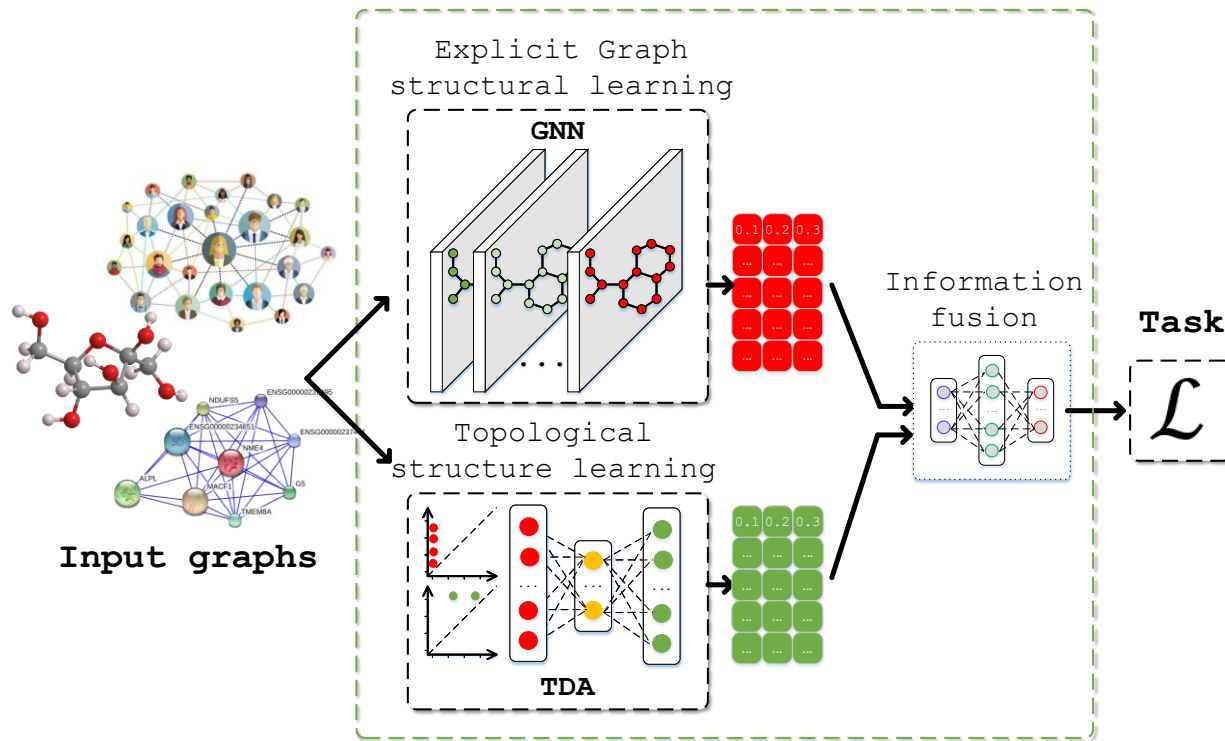


- Brain tumor: 99%
- Normal: 1%

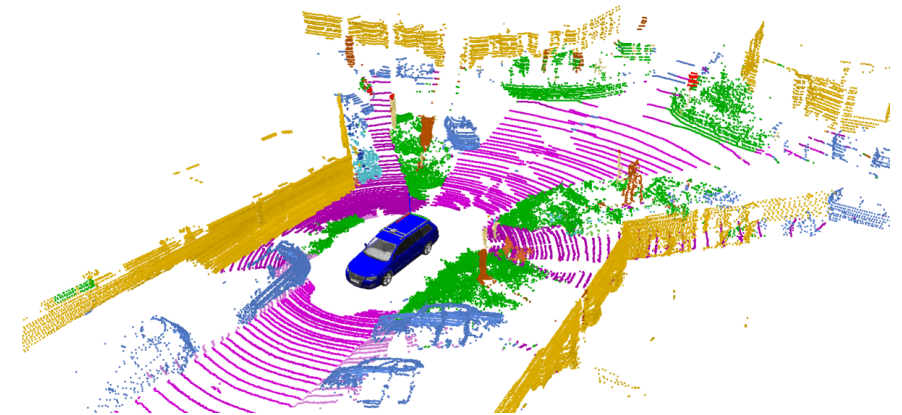
Persistence images capture global topological structural information.



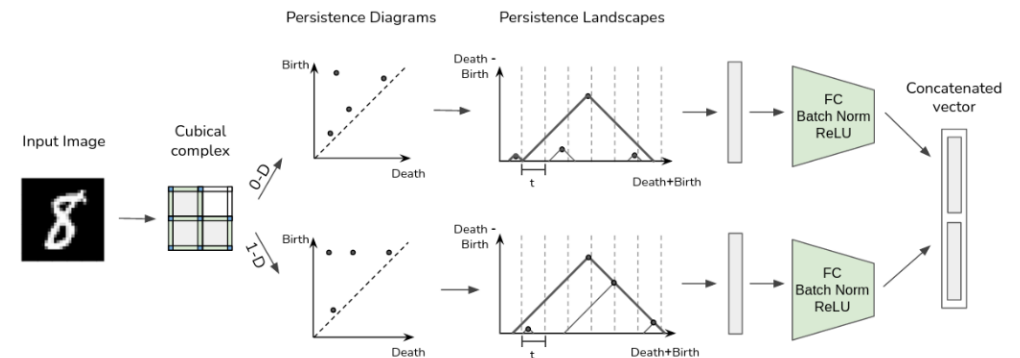
TopoDL: real-world applications



Pham, P., Bui, Q. T., Nguyen, N. T., Kozma, R., Yu, P. S., & Vo, B. (2025). Topological data analysis in graph neural networks: Surveys and perspectives. IEEE Transactions on Neural Networks and Learning Systems, 36(6), 9758-9776.



Source:
https://lewtun.github.io/hepml/lesson06_persistent-homology/



Khramtsova, E., Zuccon, G., Wang, X., & Baktashmotlagh, M. (2022, November). Rethinking persistent homology for visual recognition. In Topological, Algebraic and Geometric Learning Workshops 2022 (pp. 206-215). PMLR.

- **Fuzzy systems** enable **reasoning** under uncertainty through membership functions.
- **Neuro-fuzzy learning** optimizes **fuzzy parameters** using gradient-based training.
- **ANFIS** integrates fuzzy inference systems with neural network architectures.
- **SHAP** provides interpretable **feature attribution** for machine learning models.
- **Grad-CAM** generates visual **explanations highlighting** influential **image regions**.

Homework 1: CNN + ANFIS for Image Classification

Topic: Image Classification using CNN Feature Extractors and ANFIS-based Output Layer

Objectives:

- Understand **image preprocessing** pipelines for **MNIST** and **CIFAR** datasets.
- Build and compare different **CNN architectures** for **image classification**.
- Extract deep visual features from **CNN models** and feed them into **ANFIS**.
- Evaluate whether **ANFIS improves** interpretability and **classification performance**.

Datasets:

– **MNIST:**

- Handwritten digit classification dataset with **10 classes** from digit 0 to 9. Each image is grayscale with size **28×28 pixels**.
- <https://docs.pytorch.org/vision/main/generated/torchvision.datasets.MNIST.html>

– **CIFAR-10:**

- Natural image classification dataset with **10 classes**, including airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Each image is RGB with size **32×32 pixels**.
- <https://www.cs.toronto.edu/~kriz/cifar.html>

Homework 1: CNN + ANFIS for Image Classification (con't)

Preprocessing: image resizing if needed, pixel normalization, data augmentation for cifar-10, train/validation/test splitting

Building models:

- Build baseline models: Simple Neural Network/MLP, Basic CNN, Deeper CNN with multiple convolutional layers, Pretrained CNN feature extractor if applicable.
- Extract latent features from the CNN before the final classifier.
- Feed **CNN features** into an **ANFIS-based output layer**.
- Compare CNN-only and CNN+ANFIS architectures.

Model evaluation:

- Report accuracy, precision, recall, and F1-score.
- Analyze confusion matrix for each dataset.
- Plot training and validation loss curves.
- Compare CNN-only vs CNN+ANFIS performance.
- Discuss the interpretability benefit of fuzzy rules and membership functions.

Python/library usage: numpy, pandas, matplotlib, scikit-learn, torch/tensorflow

Homework 2: CNN + Grad-CAM for Medical Image

Topic: Medical Image Classification and Visual Explanation using CNN + Grad-CAM

Objectives:

- Understand medical image preprocessing and augmentation pipelines for ultrasound and MRI images.
- Fine-tune CNN-based models for medical image classification tasks.
- Apply Grad-CAM to visualize important image regions influencing CNN predictions.
- Compare classification performance and explanation quality across two medical datasets.

Datasets:

– **Breast Ultrasound Images Dataset (BUSI):**

- Breast ultrasound dataset containing **3 classes: normal, benign, and malignant**.
- The dataset also provides corresponding tumor mask annotations, which can be used to evaluate whether Grad-CAM focuses on meaningful lesion regions.

– **Brain Tumor Image Dataset (BTID):**

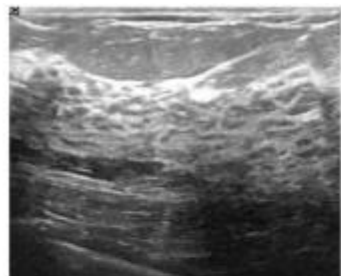
- Brain tumor image dataset. The task is to classify tumor categories and visualize model attention using Grad-CAM heatmaps.

Homework 2: CNN + Grad-CAM for Medical Image (con't)

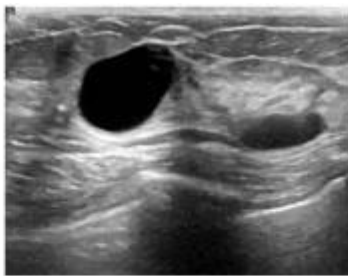
Breast Ultrasound Images Dataset (BUSI):

Al-Dhabyani, W., Gomaa, M., Khaled, H., & Fahmy, A. (2020). Dataset of breast ultrasound images. Data in Brief, 28, 104863.

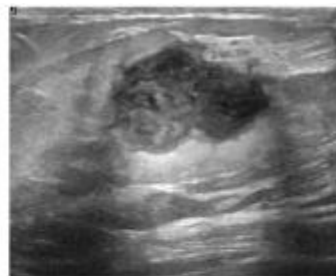
<https://www.kaggle.com/datasets/subhajournal/busi-breast-ultrasound-images-dataset>



Normal US image



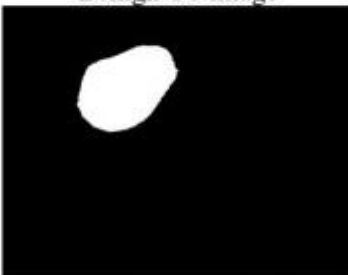
Benign US image



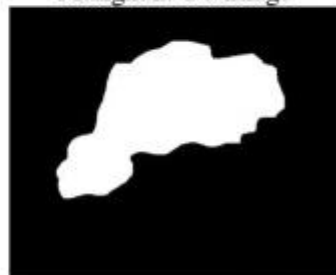
Malignant US image



Ground truth image (Normal)



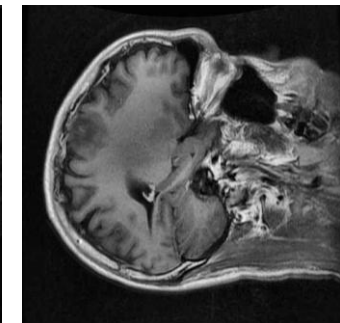
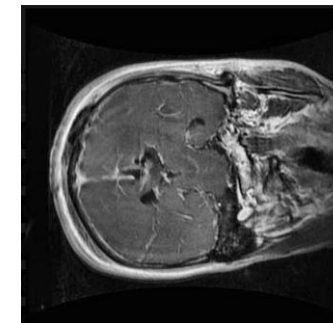
Ground truth image (Benign)



Ground truth image (Malignant)

Brain Tumor Image Dataset (BTID)

<https://huggingface.co/datasets/dwb2023/brain-tumor-image-dataset-semantic-segmentation>



Homework 2: CNN + Grad-CAM for Medical Image (con't)

Medical image preprocessing:

- Image resizing to **224×224**, RGB conversion and ImageNet normalization.
- Data augmentation: random horizontal flip, rotation, affine transformation.
- **Mask processing** for visual comparison with **Grad-CAM heatmaps**.

Building model:

- Use **CNN-based pretrained models (ResNet, VGG, etc.)** as the main CNN backbone.
- Replace the final classification layer according to the number of classes.
- Fine-tune the CNN model using cross-entropy loss.
- Compare different CNN settings, such as frozen backbone and full fine-tuning.
- **Generate Grad-CAM heatmaps** from the final convolutional layer, **overlay Grad-CAM maps** on original medical images.

Model evaluation:

- Report classification accuracy, precision, recall, and F1-score.
- Visualize Grad-CAM heatmaps for correctly and incorrectly classified samples.
- Compare Grad-CAM heatmaps with ground-truth masks or segmentation regions.
- Optionally report localization metrics such as IoU and Dice.

Python/library usage: numpy, pandas, matplotlib, scikit-learn, torch, torchvision, datasets, kagglehub, PIL.